

System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models

Sam Adè Jacobs
Microsoft Inc
Redmond, USA
samjacobs@microsoft.com

Masahiro Tanaka
Microsoft Inc
Redmond, USA
mtanaka@microsoft.com

Chengming Zhang
Microsoft Inc
Redmond, USA
v-chengmingz@microsoft.com

Minjia Zhang
Microsoft Inc
Redmond, USA
minjiaz@microsoft.com

Reza Yazdani Aminabadi
Microsoft Inc
Redmond, USA
rezayazda@microsoft.com

Shuaiwen Leon Song
Microsoft Inc
Redmond, USA
leonsong@microsoft.com

Samyam Rajbhandari
Microsoft Inc
Redmond, USA
samyamr@microsoft.com

Yuxiong He
Microsoft Inc
Redmond, USA
yhe@microsoft.com

ABSTRACT

Computation in a typical Transformer-based large language model (LLM) can be characterized by batch size, hidden dimension, number of layers, and sequence length. Until now, system works for accelerating LLM training have focused on the first three dimensions: data parallelism for batch size, tensor parallelism for hidden size, and pipeline parallelism for model depth or layers. These widely studied forms of parallelism are not targeted or optimized for long sequence Transformer models. Given practical application needs for long sequence LLM, renewed attentions are being drawn to sequence parallelism. However, existing works in sequence parallelism are constrained by memory-communication inefficiency, limiting their scalability to long sequence large models. In this work, we introduce Ulysses, a novel, portable, and effective methodology for enabling highly efficient and scalable LLM training with extremely long sequence length. Ulysses at its core partitions input data along the sequence dimension and employs an efficient all-to-all collective communication for attention computation. Theoretical communication analysis shows that, whereas other methods incur communication overhead as sequence length increases, Ulysses maintains constant communication volume when sequence length and compute devices are increased proportionally. Furthermore, experimental evaluations show that Ulysses scales to more than 1 million context length and trains 2.5x faster with 4x longer sequence length than the existing method SOTA baseline.

CCS CONCEPTS

• **Theory of computation** → **Massively parallel algorithms**; • **Computing methodologies** → **Massively parallel algorithms**; **Natural language processing**.

KEYWORDS

AI/ML, long context LLM, HPC, distributed training

ACM Reference Format:

Sam Adè Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Reza Yazdani Aminabadi, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2024. System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. In *ACM Symposium on Principles of Distributed Computing (PODC '24)*, June 17–21, 2024, Nantes, France. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3662158.3662806>

1 INTRODUCTION

Training large models with long sequences is becoming very important across the board from generative AI to models for scientific discovery. On generative AI side, conversational AI, knowledge-rich long document summarization and video generation require reasoning over long contexts in spatial and temporal domains. For example, multimodal foundation models such as ones that process speech, images and waveforms concurrently require long context reasoning over high dimensional inputs with long sequences [5, 20, 29]. Similarly, chapter and book level summarization (estimated at tens and hundreds of thousands of words) are of great importance in conversational AI and abstractive summarization tasks [1, 13, 16] and have shown to benefit from long sequence training [21, 31, 34]. The debut of ChatGPT (and subsequent similar open source and "product" LLM brands) has pushed chat application to the forefront of modern AI, making chat applications to be more relevant than ever before. Processing long sequence is crucial for supporting longer histories in chat applications [31].



This work is licensed under a Creative Commons Attribution International 4.0 License.
PODC '24, June 17–21, 2024, Nantes, France
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0668-4/24/06.
<https://doi.org/10.1145/3662158.3662806>

Long sequence length is equally critical for AI for science opening doors for better understanding of structure biology, health care, climate and weather forecasting [19] and large molecular simulation [36]. For example, by adapting large language models with gene sequences, we can create language models that can learn the evolutionary patterns of genomes using simple alphabets and extremely long sequences (the human genome has 6.4 billion letters) [36]. In health care, diagnostic predictive model conditioned on entire patient care record requires context of long sequences [9, 15].

Despite the emerging importance of long sequence length for both generative AI and AI for science, existing large model training systems and the underlying parallelism technologies (data, tensor, pipeline, sequence parallelism) are limited in their ability to support the efficient long sequence training. Two challenges with existing parallelism approach come to the fore. First, existing parallelism approach such as data, tensor and pipeline parallelism cannot address the scaling along sequence dimension. Second, existing sequence parallelism approaches are not effective because of memory-communication inefficiencies. Furthermore, existing approaches have limited usability requiring intrusive and error prone code refactoring.

In this paper, we introduce Ulysses, a novel, simple, portable, and effective methodology for enabling highly efficient and scalable LLM training with extremely long sequence lengths. Ulysses partitions individual samples along the sequence dimension among participating GPUs. Then right before the attention computation, it employs all-to-all communication collective on the partitioned queries, keys, and values such that each GPU receives the full sequence but only for a non-overlapping subset of the attention heads. This allows the participating GPUs to compute attention for different attention heads in parallel. Finally, Ulysses employs another all-to-all to gather the results along the attention heads while repartitioning along the sequence dimension.

In this work, we put forward the following contributions of Ulysses to advance state of the art in long sequence parallelism:

- We introduce a novel all-to-all communication-based sequence parallelism and its combination with model parameter sharding (e.g., ZeRO [25, 26]) for efficient training of long sequence LLMs.
- Ulysses trains Transformer models 4x larger sequence lengths than existing systems, while enabling training with sequences with more than a million tokens.
- Communication reduction of over 10x compared to existing systems, resulting in throughput improvements of up to 2.5x, and sustained throughput of over 175 TFlops/GPU (over 54% of hardware peak).
- Fully general and implementation-agnostic attention: Ulysses supports dense as well as sparse attention, and it works with efficient attention implementations such as FlashAttention v2 [5].
- Easy-to-use and portable, requiring minimal code changes to existing training frameworks.
- Open source implementation available at: <https://github.com/microsoft/DeepSpeed> and <https://github.com/microsoft/Megatron-DeepSpeed>

In subsequent sections, we provide background and related work, a detailed discussion of Ulysses core design, communication complexity analysis, experimental evaluation, and comparison with existing work.

2 BACKGROUND AND RELATED WORK

In this section, we present a brief overview of Transformer architecture, mode of parallelism to accelerate Transformer training and a discussion on closely related work to our approach.

2.1 Background

This section briefly introduces Transformer architecture and highlights different mode of parallelism of deep neural network in general and Transformer model in particular. This brief discussion is followed by specific focus on closely related work.

2.1.1 Transformer Architecture. Shown in Figure 1 is a sketch of building blocks of a typical multi-head Attention Transformer architecture [32]. It consists of input sequences which are projected into queries (Q), keys (K) and values (V) embeddings. QKV are typically a 3D tensor of size N, b, d where N is the sequence length, b is the micro batch size and d is the hidden dimension. The QKV tensors are fed to the attention block, a central component of the Transformer model. Outputs of attention are inputs to the multilayer perceptron (MLP) or position-wise feed-forward block of Transformer architecture.

The attention block followed by MLP block are replicated multiple times to form an encoder, a decoder or an encoder-decoder Transformer network.

2.1.2 Mode of Parallelism. Data parallelism [7] is de facto method of accelerating neural network training and has been applied widely with different neural network architectures and applications. Data parallelism in its simplest form partitions input data across sample or batch dimension while replicating model parameters across compute devices. Data parallelism is limited when model is large and model parameter replication across devices is practically infeasible. ZeRO [25, 26] optimization addresses this problem by partitioning model parameters across available compute devices. Both data parallelism and ZeRO do not address the problem of large intermediate activations associated with long sequences. Moreover, large batches are known to have negative impacts on model quality [11].

It's important to note that our proposed approach is orthogonal to both data parallelism and ZeRO, meaning it can be used in conjunction with both methods. Using sequence parallelism, we can maintain a fixed or reasonable global batch size on large systems, effectively mitigating the impact of large batch sizes on model convergence.

Sequence parallelism serves two purposes in this context. First, it can accelerate the time to solution for a fixed sequence length. In other words, sequence parallelism reduces the iteration time in proportion to the additional compute resources. Given more GPUs, the fixed global batch size is maintained and additional GPU resources are used for sequence parallelism. This also allows for training scenarios where the batch size per GPU (also known as micro batch size) is less than 1.

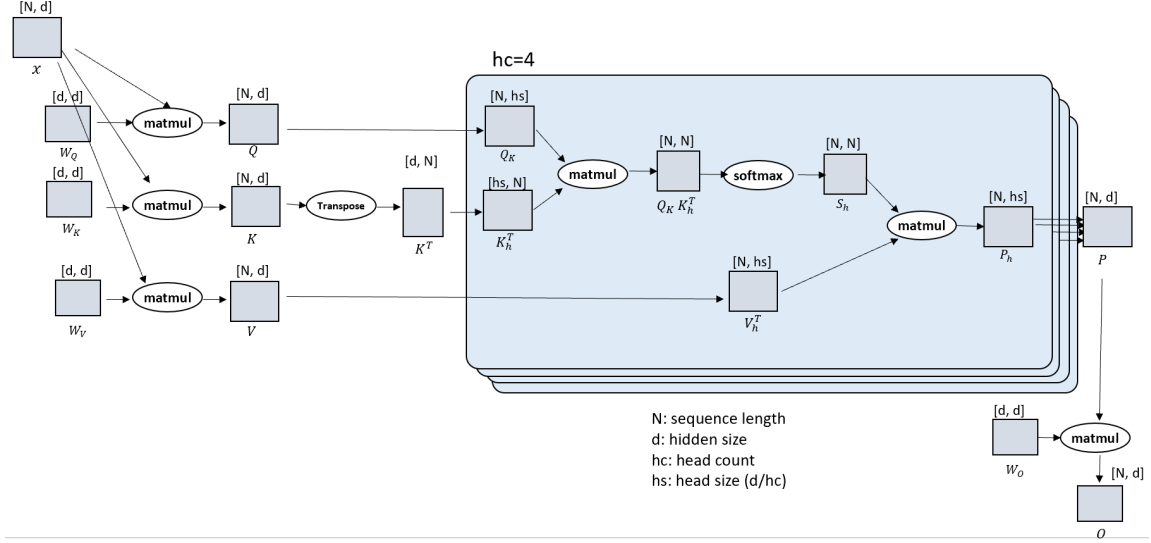


Figure 1: Multi-head attention Transformer

Second, sequence parallelism enables long sequence training or continual pretraining where the training context length gradually increases over time. Consider a real-world scenario of large-scale training on 1024 GPUs. The initial exploratory or pretraining setup of a large language model (LLM) has a sequence length of 8192 (8K) and a micro batch size of 1 per GPU, resulting in a global size of 8 million tokens. To improve the quality of the pretrained model, a simple change in sequence length from 8K to 32K is required, which would result in a global batch size of approximately 32 million. However, increasing the global batch size is not an option due to its negative impact on model quality. Therefore, sequence parallelism comes in handy as a system optimization technique that does not require laborious hyperparameter search. In this scenario, sequence parallelism allows 32K tokens to be split across multiple GPUs along the sequence dimension (e.g., a sequence parallel group of 4) without increasing the global batch size.

Tensor [27] and pipeline parallelism [10, 17, 18] are two other popular methods for large-scale training. Collectively, tensor and pipeline parallelism are called model parallelism, and are targeted at compute operators in large models. In contrast to data parallelism, model parallelism are used when models are too large (as it is in many LLMs) and cannot be fully replicated across data parallel ranks. Tensor parallelism splits compute operators (i.e. attention and MLPs) within a layer, and pipeline parallelism splits model in a depth-wise (layer-wise) fashion. 3D parallelism [28, 30] combines data parallelism, tensor parallelism, and pipeline parallelism to achieve higher throughput compared to the three constituent components, at the cost of extensive code rewrite and productivity overhead [33].

2.1.3 Long Sequence Applications in Scientific Machine Learning (SciML). Many real-world science models need long-sequence support. For example, GenSLM [36] is one of the first whole-genome scale foundation models that can be generalized to other prediction tasks. GenSLMs utilization of genome-scale language

models that learn the evolutionary landscape of SARS-CoV-2 genomes. GenSLMs can identify and classify new and emerging variants of pandemic-causing viruses, particularly SARS-CoV-2. To have a good understanding of the latent space, GenSLMs, and similar models require very long sequence support for both training and inference.

Traditional weather and climate modeling approaches are based on physics-informed numerical models of the atmosphere, which are computationally intensive and challenging due to the non-linear dynamics and complex interactions between variables. ClimaX [19] is developed as a flexible and generalizable deep-learning model for weather and climate science. It can be trained using heterogeneous datasets that span different variables, spatio-temporal coverage, and physical groundings. The model extends the Transformer architecture, incorporating novel encoding and aggregation blocks that enable the effective use of computing resources while maintaining utility. However, current architectures struggle to support inputs with global resolution beyond 3 km, unless model parallelism is employed.

2.2 Related Work

For a broad overview and survey of distributed training methods for deep neural networks please see [2]. These methods are broadly categorized into data and model parallelism as described above. However, all of existing parallel methods are limited in dealing with intermediate activation memory overhead associated with extremely long sequence.

While recent works in sequence parallelism address the memory overhead, they are lacking in communication efficiency, thus limited in scaling capability. Similarly to our work, all existing works in sequence parallelism partition the input data along sequence dimension but differ in what input projections are partitioned and how partitions are aggregated and communicated for attention computation.

Table 1: Comparison of our work (Ulysses) to other sequence parallelism methods.

Methods	Comm complexity	Activation memory efficiency	Parameter memory efficiency	Attention agnostic	Ease of use
ColAI-SP [14]	$O(M)$	✓	✗	✗	✗
Megatron-SP [12]	$O(M)$	✓	✓	✓	✗
Ulysses	$O(M/P)$	✓	✓	✓	✓

The authors in [14] (henceforward referenced as *ColAI-SP*) introduce ring self-attention, a ring-like communication collective in which query projections are local whereas key and values projections are transmitted in a ring-style to compute global attention, resulting in communication complexity linear in message size, M .

Megatron-LM sequence parallelism [12] approach is tightly integrated with Megatron tensor parallelism. Megatron-LM partitions sequences along sequence dimensions and applies the all-gather and reduce-scatter collective to aggregate QKV projections for attention computation. Communication complexity analysis shows that, unlike our approach, Megatron-LM sequence parallelism increases the communication time linearly with message size (M) regardless of the number of compute devices. Ulysses, on the other hand, keeps communication time constant by increasing GPUs proportional to the message size or the sequence length. In general, with our all-to-all based approach, both the source and target data on each GPU are M/P . However, in allgather, the source data is M/P while the target data is M . The amount of data that each GPU receives from another in all-gather is P times larger than in all-to-all. Thus, Ulysses leads to a smaller amount of data transfer than all-gather in Megatron-LM (see subsection 3.2 for more details).

Table 1 summarizes how Ulysses differs from other existing methods. Ulysses has communication efficiency advantage over the other two methods. It also benefits from extending ZeRO [25, 26] optimization for model parameter partitioning across both sequence and data parallel groups. Ulysses supports different kinds of attention and it is easy to use. Megatron-LM sequence parallelism is tightly integrated with its tensor parallelism, limiting both its memory efficiency and ease of use. *ColAI-SP* requires a different (specific) kind of attention and is not easy to use. It is not clear how well *ColAI-SP* ring self-attention generalizes to other types and mechanisms of attention.

There are related works in sparse Transformers, particularly focusing on full-attention approximation such as sparse attention [1, 3, 4, 35]. There are also recent works [5, 6, 23] on single GPU memory and compute efficient attention. A popular example in this category is FlashAttention [5, 6], which leverages known techniques such as tiling and recomputation to optimize memory and compute efficiency. These works are orthogonal and complementary to our work and can be leveraged by Ulysses.

3 ULYSSES

3.1 Core System Design

Figure 2 shows the core design of Ulysses. As in the known transformer architecture, the design consists of input sequences

of size N partitioned across P available devices. Each local N/P partition is projected into queries (Q), keys (K) and values (V) embeddings. Next, (QKV) embeddings are gathered into global QKV through highly optimized all-to-all collectives between the participating compute devices. Sequel to all-to-all collective is the attention computation per head in the form:

$$\text{Output context} = \text{Softmax}((QK^T)/\sqrt{d})V \quad (1)$$

After the attention computation, another all-to-all collective transforms output context tensor of attention computation to sequence (N/P) parallel for subsequent operators (MLP MatMul, layer norm etc) in the remaining modules of transformer layer block. Essentially, the computation pattern of Ulysses transitions from sequence parallelism to head parallelism, and then back to sequence parallelism. Efficient all-to-all communication is crucial for transitioning between these two modes of parallelism.

3.2 Communication Analysis

What distinguishes Ulysses from the other existing long-sequence approaches is our much smaller aggregate communication volume and overall better scalability with increasing degree of sequence parallelism compared to existing solutions, as demonstrated by the communication volume analysis below:

On modern clusters with intra-node NVSwitch interconnect and inter-node fat tree IB topology, the communication volume transmitted per link for an all-to-all for aggregate message of size M over P GPUs is M/P . For a transformer model with hidden size d , sequence length of N , and parallelism degree of P , Ulysses performs all-to-all for the QKV projections with an aggregate message size of $3Nd$ before the attention computation, and another all-to-all for output context projection with a size Nd for each transformer layer. Therefore, Ulysses incurs an aggregate communication volume per link of $4Nd/P$ (or with the complexity of $O(N/P)$). Note that this communication volume is constant when both N and P are increased proportionally.

In contrast, existing approaches such as the Megatron-LM [12] and ColAI [14] sequence parallelism incur a communication volume that increases linearly with N regardless of P , resulting in the communication complexity of $O(N)$. For instance, Megatron-LM conducts an all-gather prior to computing QKV and an reduce-scatter after attention computation. Furthermore, it needs additional all-gather and reduce-scatter for an MLP block. The cost of each all-gather and reduce-scatter remains constant at size M , regardless of P . As a result, the communication cost for Megatron-LM's sequence parallelism is $4Nd$ for each Transformer layer, which is P times larger than that of Ulysses. Consequently, Ulysses enables training with extremely long sequences while achieving significantly higher training efficiency compared to the existing approaches.

Table 2: Ulysses vs Megatron-LM communication collective benchmark on 8 A100 GPUs

Methods	Sequence Length	Hidden Size	Reduce Scatter	All Gather	AllTo All	Total Comm Time
Megatron-LM	256K	4K	4GB	4GB	NA	34ms
Ulysses	256K	4K	NA	NA	8GB	4.9ms

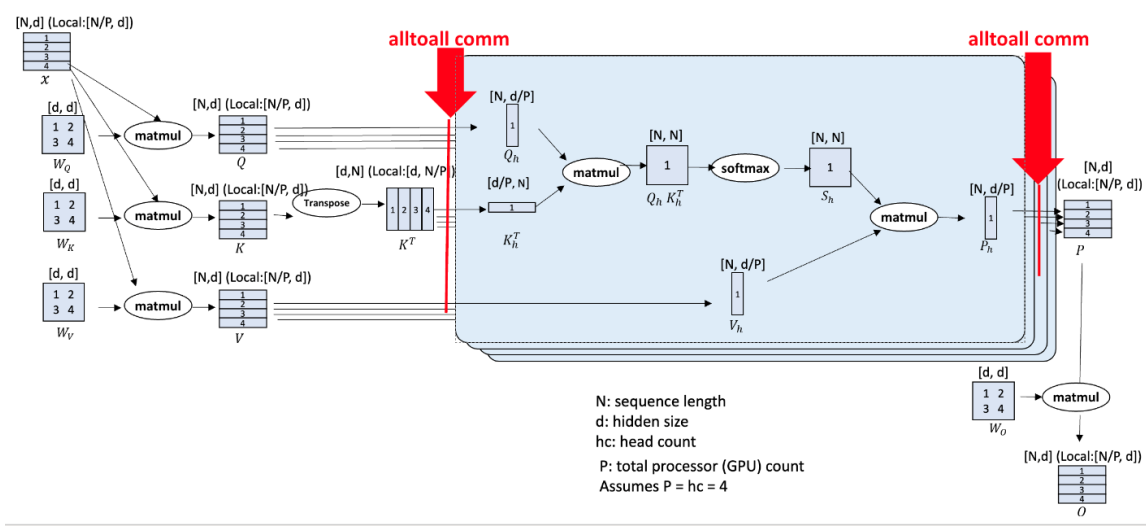


Figure 2: Ulysses core design

Table 2 shows an empirical microbenchmark evaluation of all-gather/reduce-scatter and all-to-all, communication primitive for Megatron-LM and Ulysses respectively. For sequence length of 256K and hidden size of 4K, Megatron-LM transmits 4GB of data for each all-gather and reduce-scatter operation across both the attention and MLP blocks, accumulating to a total data transfer of 8GB. In contrast, Ulysses transfers 8GB of data (that is, for $4Nd$ shown previously). However, when comparing the communication cost for the same message size, the all-to-all operation is 7x faster than the combined all-gather/reduce-scatter operations. The reason for the faster data transfer in all-to-all is that each GPU pair involved only has to send a fraction, $1/P$, of the total amount of data, without contention (see Figure 3).

3.3 Additional Benefits

3.3.1 Support for Large Models. While Ulysses reduces the activation memory when training with longer sequences, it does not impact the memory consumed by the model states. Therefore, to support large sequence length training with a large language model, Ulysses is integrated with ZeRO Redundancy Optimizer (ZeRO) [25, 26].

ZeRO is a memory optimization technique for training large models. Unlike the classic data parallel training of neural networks where model states are replicated across data parallel ranks, ZeRO (ZeRO Stage 3 or ZeRO-3 in particular) optimizes memory usage by partitioning model states across data parallel ranks. However, with sequence parallelism, training data can be considered in both batch (sample) and sequence dimensions and the associated parallel groups combined to form a larger group for ZeRO parallelism. Therefore, we extend ZeRO partitioning to combination of data parallel and sequence parallel ranks. In other words, in Ulysses, ZeRO partitions model states across both sequence and data parallel group and collects per-rank partitions (allgather) when they are needed. Similarly, gradients are reduced across both data and sequence parallel ranks for parameter update. ZeRO support allows

for huge memory savings in both sequence and data dimensions and enables scaling not just to large sequence lengths but also to large models.

3.3.2 Generality. Ulysses implementation of distributed attention module is general enough to support any attention: e.g., self-attention, cross-attention, causal attention in both their dense and sparse counterparts, and their various optimized kernels that support long-sequence at local attention level such as different versions of FlashAttention. The general property of Ulysses stems from the modular nature of its core design: an attention-centric sequence parallelism design. Prior to attention computation is sequence parallelism of N/P partition, attention computation is head parallelism with full attention per head but just with fewer heads, thus attention computation can be replaced with any type of attention mechanisms, e.g., dense attention and various forms of sparse attention.

4 EVALUATION

We conducted our experiments on the ThetaGPU Cluster at the Argonne National Laboratory. This cluster is built with NVIDIA DGX A100 nodes. Each node is equipped with two AMD Rome CPUs and eight NVIDIA A100 GPUs, each with 40GB of memory. These nodes utilize high-performance NVLink and NVSwitch technologies for intra-node communication. The interconnect system features eight ports of HDR200 InfiniBand, delivering an impressive aggregated bandwidth of 200GB/s.

As for model configurations, we employed the standard GPT architecture [24], but with varying sizes. For instance, the GPT-7B model has a hidden dimension of 4096, 32 heads, and 32 layers. On the other hand, the GPT-30B model has a hidden dimension of 6144, 64 heads, and 64 layers.

Our evaluations are six-fold: i) sequence length scalability, ii) throughput for dense attention and comparison with existing sequence parallelism methods, and iii) throughput with sparse

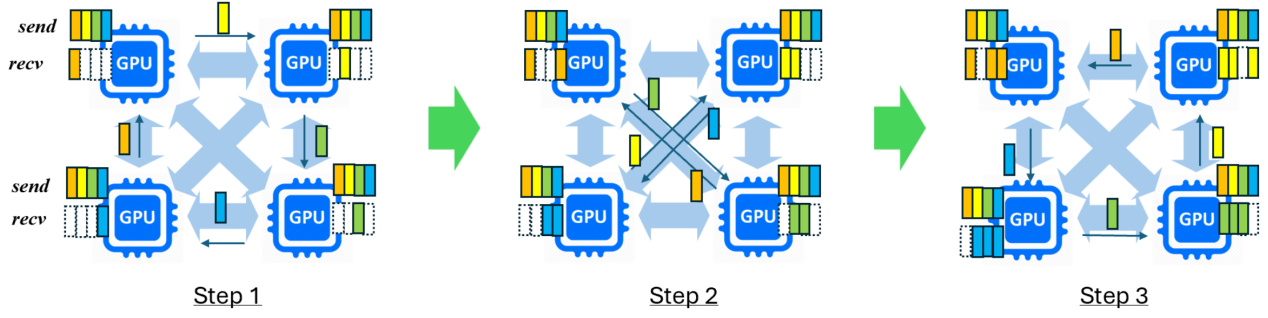


Figure 3: All-to-all collective communication illustration for our method. Each GPU (rank) sends $1/P$ of total volume at each step without contention. States of receive buffer while messages are in transmission are shown in dotted lines.

attention and comparison with existing sequence parallelism methods, iv) comparative analysis of sequence length capabilities of ZeRO and Ulysses, v) parallel scaling study and vi) convergence study of Ulysses. We discuss and present evaluations from each of these categories next.

4.1 Sequence Length Scalability

The first set of experiments is a strong scaling of sequence length up to 1 million tokens on the 1.2 billion-parameter GPT model (GPT-1.2B). The results of this evaluation are shown in Figure 4. Ulysses allows increasing sequence length linearly with the number of GPUs and sequence length scales linearly relative to and maintains similar computation throughput across different sequence lengths at appropriate GPU count.

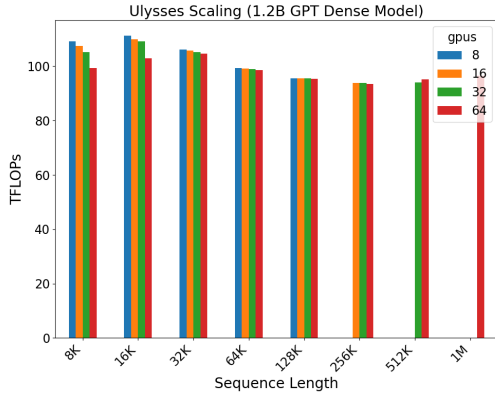


Figure 4: Ulysses strong scalability evaluation of GPT-1.2B at different sequence length and GPU counts

4.2 Dense Attention Evaluation

Next, we evaluated Ulysses on 7 billion (7B) and 30 billion (30B) parameter GPT dense attention models and compared against Megatron-LM’s sequence parallelism (Megatron LM) and Colossal AI sequence parallelism (ColAI-SP) on 32 and 64 A100 GPUs, respectively. For fair comparison, we applied appropriate

model parallelism in other methods: tensor parallelism (TP) with Megatron-LM sequence parallelism and ZeRO-3 parameter sharding with ColAI-SP. The results of our evaluations are shown in Figures 5 and 6.

We compared Ulysses with Megatron-LM and ColAI-SP for 7B and 30B models running various sequence lengths. We chose the sequence parallelism degree and micro-batch size that produced the best performance (measured as TFLOPs) for the three methods, this we call optimal (batch size-sequence length) configurations. In other words, for each fixed sequence length and GPU count, and for each of the three methods (Ulysses, Megatron-LM, and ColAI-SP), we did a sweep of batch size, data parallel degree or tensor parallel degree, and sequence parallel degree and report the highest achievable TFLOPs or OOM (in scenario where sequence length doesn’t fit). For Ulysses, we always use a ZeRO-3 parallelism degrees of 32 and 64 for 7B and 30B models respectively.

Figures 5 and 6 show that Ulysses consistently outperforms Megatron-LM and ColAI-SP for the sequence length that can be run with them. In addition, Ulysses can run longer sequence than the two existing methods. Ulysses performance advantages are two folds: (1) Ulysses in combination with ZeRO-3 parameter sharding across both data and sequence parallel groups fits more samples than Megatron-LM and ColAI-SP because of the memory optimization leading to higher throughput (2) Ulysses benefits from efficient all-to-all communication relative to all-gather reduce-scatter and ring-style P2P communication as applied in Megatron-LM and ColAI-SP sequence parallelism. However, for dense attention at long sequence length, the throughput is primarily determined by local attention computation due to quadratic computation complexity of attention; therefore, performance gap between Ulysses and the two existing methods closes for sequence length that can be run with them.

4.3 Sparse Attention Evaluation

Similarly, we evaluated Ulysses on 7 billion and 30 billion parameter sparse attention models and benchmarked against Megatron-LM sequence parallelism. There is no public implementation of block sparse attention for ColAI-SP, therefore, evaluation of sparse attention is in comparison with Megatron-LM. Results of our evaluation are shown in Figures 7 and 8. We observed similar trends with sparse attention as in dense attention experiments.

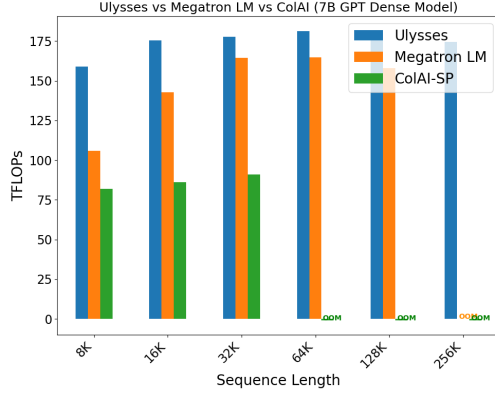


Figure 5: Evaluation of Ulysses vs Megatron LM vs ColAI-SP on GPT-7B parameter model with dense attention (32 GPUs)

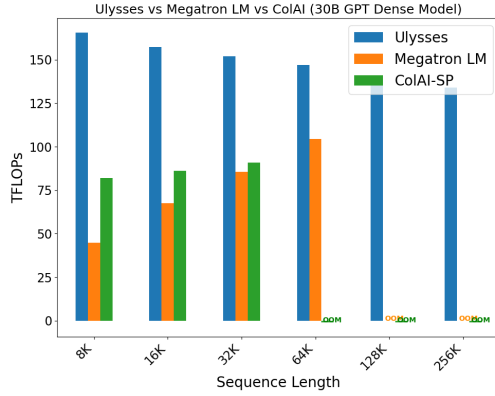


Figure 6: Evaluation of Ulysses vs Megatron LM vs ColAI-SP on GPT-30B parameter model with dense attention (64 GPUs)

We observed more than 2x throughput performance of Ulysses compared to Megatron-LM. Similarly for communication reduction, you may recall communication complexity of Ulysses as $O(M/P)$ compared to $O(M)$ of Megatron-LM, where M is message size and P is number of GPUs. Ulysses performs all-to-all after computing QKV resulting in 3 more volume than Megatron. With P set to 32 for GPT-7B model, $3M/32$ is approximately 10x in communication reduction compared to Megatron-LM. For memory saving, Ulysses combined with ZeRO-3 scales to 4x longer sequence lengths than Megatron-LM.

Ulysses outperforms Megatron-LM for sequence length that can be run with both. In fact, the current Ulysses throughput is bottlenecked by the local sparse attention implementation, and as a result Ulysses throughput decreases as the sequence length increases. We expect this gap in performance between our method and Megatron-LM to increase further for larger sequence lengths as we improve the performance of the local sparse attention implementation in the future. A noteworthy observation is that the decreasing performance gap between Ulysses and Megatron-LM observed in dense attention evaluation is less pronounced in sparse

attention evaluation, because the attention computation in sparse attention is less dominant compared to dense attention.

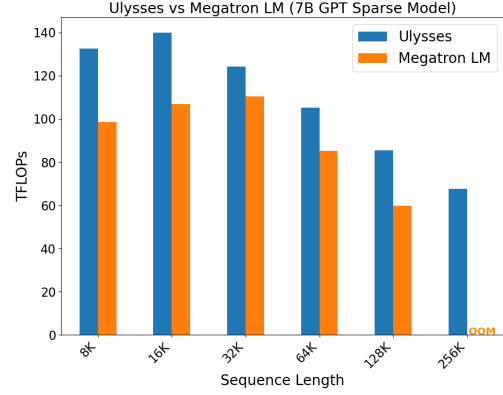


Figure 7: Evaluation of Ulysses and Megatron LM on GPT-7B parameter model with block sparse attention (32 GPUs)

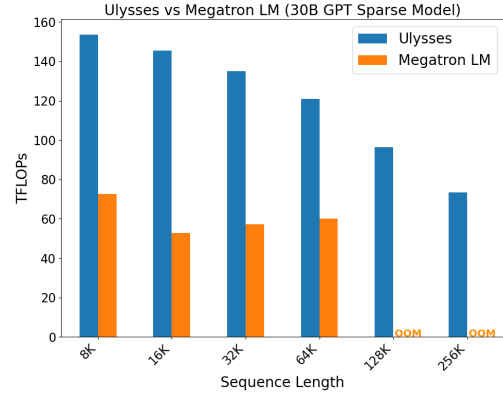


Figure 8: Evaluation of Ulysses and Megatron LM on GPT-30B parameter model with block sparse attention (64 GPUs)

4.4 Ulysses and ZeRO: A Comparative Analysis of Sequence Length Capabilities

We conducted an additional experiment to quantify performance benefits of Ulysses relative to ZeRO-3 data parallelism. Remember that ZeRO-3 partitions model weights, while Ulysses partitions sequences. Although ZeRO-3 enables training with large models, it has a significant limitation when training with long sequences due to the large activations associated with these sequences. As shown in Table 3, when using only ZeRO-3, the maximum sequence lengths achievable for training with the GPT-7B and GPT-30B dense models are 16K and 8K, respectively. However, these maximum lengths increase to 256K when ZeRO-3 is combined with Ulysses.

Table 3: ZeRO and Ulysses: A comparative analysis of sequence length capabilities

Methods	Model Size	GPUs	Max Seqen	TFLOPs at Max Seqen
Ulysses	7B	32	256K	174.49
ZeRO	7B	32	16K	175.3
Ulysses	30B	64	256K	134.03
ZeRO	30B	64	8K	165.52

Table 4: Ulysses parallel scaling study with fixed sequence length

Seqen	GPUs	Time (ms)	TFLOPs
128K	64	32432.1333	165.526
128K	128	17052.5143	157.41
128K	256	9886.7	136.09

Table 5: Ulysses parallel scaling study with varying sequence length

Seqen	GPUs	Time (ms)	TFLOPs
64K	64	9676.76	161.362
128K	128	17052.5143	157.41
256K	256	33486.5	147.4

4.5 Parallel Scaling Study

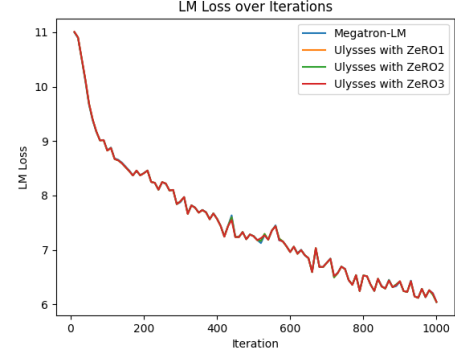
Furthermore, we conducted parallel scaling studies of Ulysses along two axes. First, we fixed the sequence length to 131,072 tokens and increased the GPU count from 64 to 256. Second, we increase the GPU count proportionally to the increase in the sequence length. The results of these experiments are shown in Tables 4 and 5 respectively. For both evaluations, we used the GPT-7B dense model with a global batch size of 8. The tables show iteration time in microseconds as well as the achieved throughput measured in per GPU TFLOPs. Table 4 can be interpreted as strong scaling and shows that execution time decreases almost linearly as we increase the GPU count. Table 5 on the other hand, is a form of weak scaling (not in the traditional sense) with caveat that attention computation, a function of sequence length, is quadratic in complexity. In other words, as we increase the sequence length, the work increases quadratically.

Communication overhead can be attributed to a slight decrease in throughput as we increase communication workload (that is, sequence length or GPU count). This overhead notwithstanding, we observe good scaling at high percentages of theoretical peak GPU performance across the two studies. These good scaling results indicate good parallel efficiency of Ulysses.

4.6 Convergence Study

Lastly, Figure 9 shows the convergence of a 1.3 billion GPT model with 32K sequence length on 8 A100 GPUs with the degree of sequence parallelism set to 4 for both the Ulysses and Megatron-LM sequence parallelism. For Ulysses, we evaluated the convergence

with different ZeRO stages. Ulysses is a purely system optimization technique that enables training of long sequence Transformer model, thus there is no (negative) on quality of trained models, this assertion is validated through experiments and is shown in Figure 9.

**Figure 9: Convergence evaluation of Ulysses with different ZeRO memory optimization stages**

5 CASE STUDY: TRAINING REAL-WORLD SCIENCE MODELS WITH ULYSSES

In this section, we demonstrate how Ulysses can be integrated in training real-world AI4Science models with long sequences requirements. Specifically, we will use an open source climate model, named *ClimaX* [19], to showcase long-sequence training for higher-resolution images in time series. ClimaX is one of the first foundation models for weather and climate modeling tasks. It can absorb a variety of datasets with diverse variables and resolutions, leading to enhanced weather prediction accuracy.

ClimaX employs Vision Transformers (ViT) [8] as its backbone architecture and treats forecasting image modeling as a sequence-to-sequence problem. Specifically, an image of shape $[H \times W \times C]$ is divided into T patches, each of size $[P_H \times P_W \times C]$. Here, H and W are the height and width of the image, and C is the number of channels. Each patch is reshaped into a vector (corresponding to a word in language problems) of length $(P_H \times P_W \times C)$ and linearly projected onto a D -dimensional vector using a trainable embedding matrix, where D is the desired dimension of the patch embedding. As a result, a 256×256 resolution image is divided into 16×16 patches with patch size 16×16 , yielding a sequence of 256 image tokens. This sequence is then fed as input to the ViT for training.

Additionally, from the aspect of climate science, achieving highly accurate and localized weather forecasts necessitates one to resolve atmospheric dynamics at sub-kilometer spatial scales. To achieve such detailed forecasts across the globe, weather foundation models need to process extremely high-resolution images with dozens of channels, thereby leading to inputs with very large sequence lengths. For example, a target is to support inputs with a global resolution greater than 3 km, corresponding to the image size of 6000×12000 . However, processing high-resolution images with the ViT model presents significant system challenges. Unlike

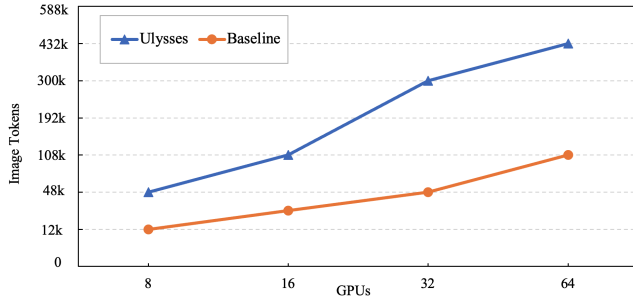


Figure 10: Evaluation of Ulysses and Baseline on the ClimaX model. The corresponding image sizes of image tokens of 12k, 48k, 108k, 192k, 300k, 432k, 588k are $256 \times 256 \times 48$, $512 \times 512 \times 48$, $768 \times 768 \times 48$, $1024 \times 1024 \times 48$, $1280 \times 1280 \times 48$, $1536 \times 1536 \times 48$, $1792 \times 1792 \times 48$, respectively.

natural language tasks that deal with 1D data, increasing the resolution of image inputs quadratically increases the image token sequence length. For instance, the image sequence length of a higher-resolution 4096×4096 image can result in a sequence with 64K image tokens. Given that memory complexity with respect to the number of tokens in the self-attention layers in ViT models also increases quadratically, higher-resolution images can easily cause out-of-memory issues.

To address this challenge, we have integrated Ulysses into ClimaX model training, which is built on top of PyTorch Lightning [22]. Figure 10 shows the maximum sequence length comparisons in the number of image tokens between Ulysses and the PyTorch Lightning baseline (with ZeRO stage 3 enabled) with the increasing number of GPUs. The original ClimaX model could only train $768 \times 768 \times 48$ images (equivalent to the sequence length of 108K image tokens) with a batch size of 1 on 64 NVIDIA A100 GPUs. Further increasing the image resolution resulted in an out-of-memory (OOM) issue. With Ulysses, we managed to reduce ClimaX’s peak memory consumption under the same image resolution. More importantly, thanks to Ulysses’ critical memory saving techniques, we are now able to train ClimaX with even higher resolutions, up to $1536 \times 1536 \times 48$ (equivalent to sequence length of 432K image tokens), 4x longer than the baseline implementation.

6 CONCLUSION

This paper presents Ulysses, a system optimization technique that enables the training of large transformer models with extremely long context length. Ulysses parallelizes Transformer along the sequence dimension and employs a novel, communication-efficient all-to-all collective for attention computation. Additional technical contributions of Ulysses are threefold: (1) novel integration with ZeRO-style data parallelism for memory reduction (2) generalized support for all general purpose and commonly used attention mechanism and (3) easy-to-use open-source implementation with adoption in both academic and industrial production workloads.

Ulysses is the first of its kind to scale to a context length of 1 million tokens with sustainable throughput at fractions of hardware peak. Training with Ulysses allows both model size and

sequence length to scale near indefinitely, unbounded by single GPU memory limitations. We present evaluation results that span both traditional LLM and emerging science applications. Furthermore, our experimental evaluation results show that our approach trains 2.5x faster with a 4x longer sequence than the existing SOTA baseline.

Future work will optimize the communication collective for sparse attention and apply our method to emerging new applications in LLMs and other domains where processing long-range dependencies are crucial to model quality.

ACKNOWLEDGMENTS

We would like to express our gratitude to Olatunji Ruwase, Heyang Qin, Stas Bekman, and Connor Holmes for their valuable discussions, as well as to all the DeeSpeed members who contributed to the release of the Ulysses software.

REFERENCES

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* [cs.CL]
- [2] Tal Ben-Nun and Torsten Hoeftler. 2019. Demystifying parallel and distributed deep learning: An in-depth concurrency analysis. *ACM Computing Surveys (CSUR)* 52, 4 (2019), 1–43.
- [3] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *CoRR abs/1904.10509* (2019).
- [4] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Szilvák, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. 2020. Rethinking Attention with Performers. *CoRR abs/2009.14794* (2020). [arXiv:2009.14794](https://arxiv.org/abs/2009.14794)
- [5] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv:2307.08691* [cs.LG]
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135* [cs.LG]
- [7] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. *Advances in neural information processing systems* 25 (2012).
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021*. OpenReview.net.
- [9] Shang Gao, Mohammed Alawad, M. Todd Young, John Gounley, Noah Schaefferkoetter, Hong Jun Yoon, Xiao-Cheng Wu, Eric B. Durbin, Jennifer Doherty, Antoinette Stroup, Linda Coyle, and Georgia Tourassi. 2021. Limitations of Transformers on Clinical Text Classification. *IEEE Journal of Biomedical and Health Informatics* 25, 9 (2021), 3596–3607. <https://doi.org/10.1109/JBHI.2021.3062322>
- [10] Yanping Huang, Yonglong Cheng, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, and Zhifeng Chen. 2018. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *ArXiv abs/1811.06965* (2018).
- [11] Nitish Shirish Keskar, Dhruvatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. 2016. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836* (2016).
- [12] Vijay Korthikanti, Jared Casper, Sangkug Lynn, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *arXiv:2205.05198* [cs.LG]
- [13] Wojciech Kryściński, Nazneen Rajani, Divyansh Agarwal, Caiming Xiong, and Dragomir Radev. 2022. BookSum: A Collection of Datasets for Long-form Narrative Summarization. *arXiv:2105.08209* [cs.CL]
- [14] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2022. Sequence Parallelism: Long Sequence Training from System Perspective. *arXiv:2105.13120* [cs.LG]
- [15] Yikuan Li, Ramsey M. Wehbe, Faraz S. Ahmad, Hanyin Wang, and Yuan Luo. 2022. Clinical-Longformer and Clinical-BigBird: Transformers for long clinical sequences. *CoRR abs/2201.11838* (2022). [arXiv:2201.11838](https://arxiv.org/abs/2201.11838)

- [16] MosaicML. 2023. Introducing MPT-7B: A New Standard for Open-Source, Commercially Usable LLMs. <https://www.mosaicml.com/blog/mpt-7b>.
- [17] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Granger, Phil Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *ACM Symposium on Operating Systems Principles (SOSP 2019)*.
- [18] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*. PMLR, 7937–7947.
- [19] Tung Nguyen, Johannes Brandstetter, Ashish Kapoor, Jayesh K. Gupta, and Aditya Grover. 2023. ClimateX: A foundation model for weather and climate. *arXiv:2301.10343* [cs.LG]
- [20] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL]
- [21] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. YaRN: Efficient Context Window Extension of Large Language Models. *arXiv:2309.00071* [cs.CL]
- [22] PyTorch. 2023. Lightning AI. <https://lightning.ai/>
- [23] Markus N. Rabe and Charles Staats. 2021. Self-attention Does Not Need $O(n^2)$ Memory. *CoRR* abs/2112.05682 (2021). *arXiv:2112.05682* <https://arxiv.org/abs/2112.05682>
- [24] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. (2019).
- [25] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [26] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*.
- [27] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv:1909.08053* [cs.CL]
- [28] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model. *arXiv preprint arXiv:2201.11990* (2022).
- [29] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. 2020. Long Range Arena: A Benchmark for Efficient Transformers. *arXiv:2011.04006* [cs.LG]
- [30] DeepSpeed Team and Rangan Majumder. 2020. DeepSpeed: Extreme-scale model training for everyone. <https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/>.
- [31] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288* [cs.CL]
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.
- [33] Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. 2023. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training. *arXiv:2306.10209* [cs.DC]
- [34] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oğuz, Madian Khabsa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. 2023. Effective Long-Context Scaling of Foundation Models. *arXiv:2309.16039* [cs.CL]
- [35] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2021. Big Bird: Transformers for Longer Sequences. *arXiv:2007.14062* [cs.LG]
- [36] Maxim Zvyagin, Alexander Brace, Kyle Hippe, Yuntian Deng, Bin Zhang, Cindy Orozco Bohorquez, Austin Clyde, Bharat Kale, Danilo Perez-Rivera, Heng Ma, et al. 2022. GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics. *bioRxiv* (2022), 2022–10.